

어텐션 메커니즘 (Attention Mechanism)

기계 번역의 패러다임 전환:
Seq2Seq의 한계 극복부터
PyTorch 실전 구현까지



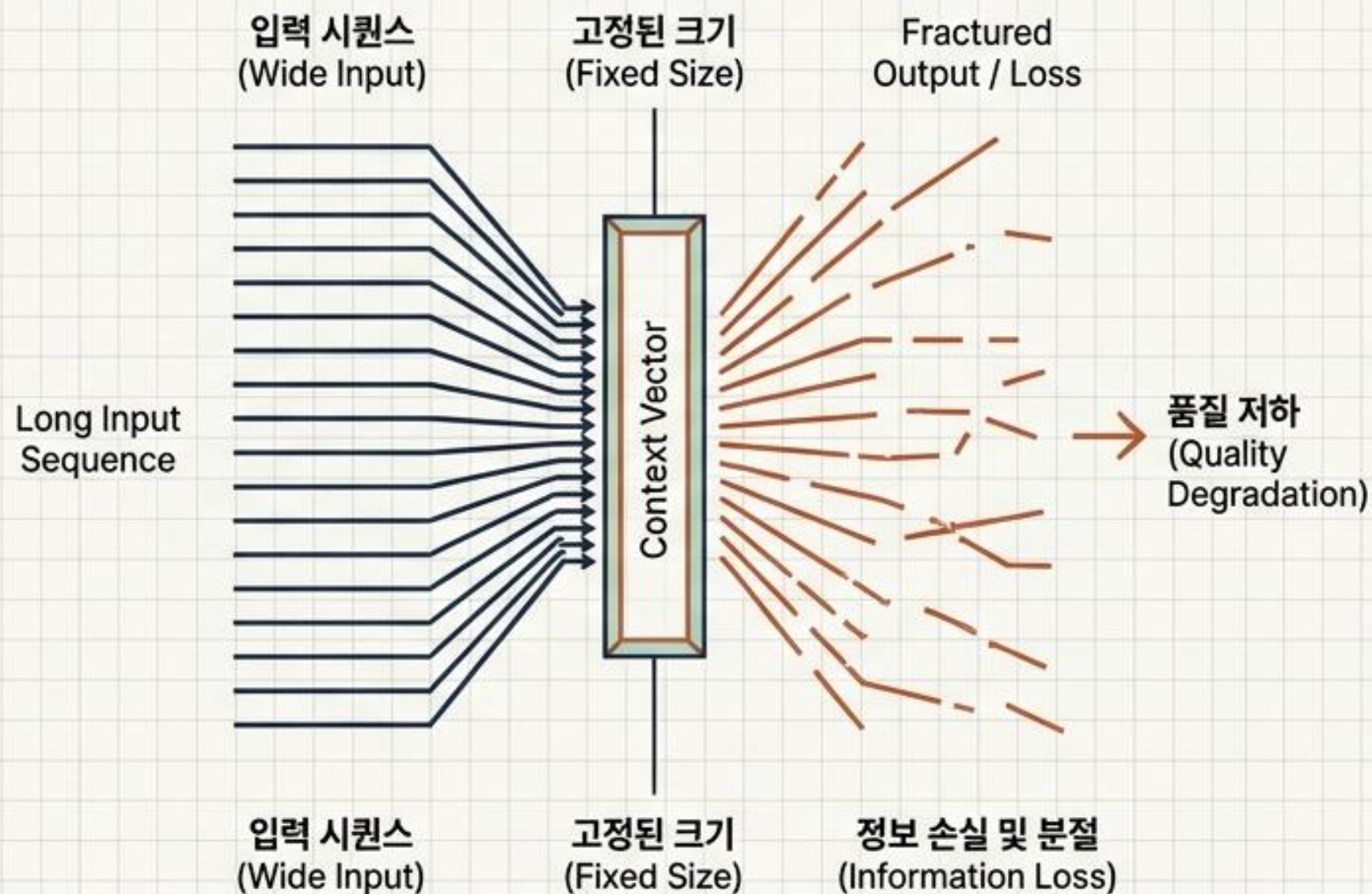
기존 Seq2Seq 모델의 치명적 한계

정보 손실 (Information Loss)

- 입력 시퀀스를 고정된 크기의 단일 '컨텍스트 벡터'에 모두 압축
- 문장 길이가 길어질수록 번역 품질이 급격히 저하되는 근본적 원인

기울기 소실 (Vanishing Gradient)

- RNN 아키텍처 자체의 고질적인 한계점
- 장기 의존성(Long-term dependency) 문제로 초기 입력 정보가 유실됨



패러다임의 전환: 어텐션(Attention)의 도입

동적 문맥 참조

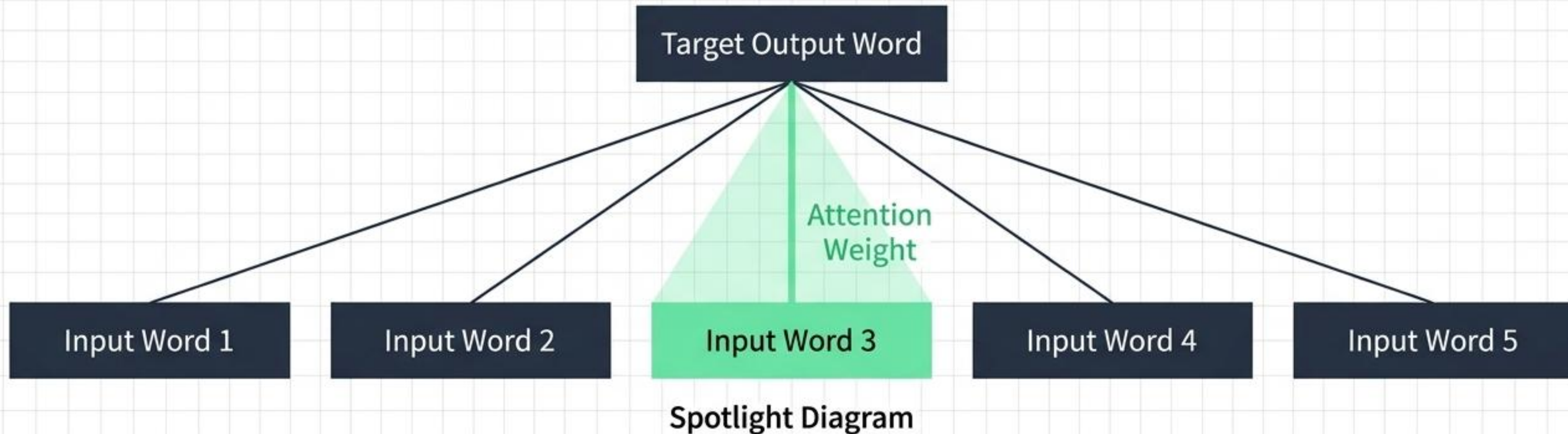
- 디코더 예측 시점마다 인코더의 전체 입력 문장을 재참조

가중치 부여 (집중)

- 예측해야 할 단어와 연관성이 높은 입력 단어에 더 높은 가중치 부여

결과적 이점

- 시퀀스 길이에 구애받지 않는 안정적 정확도 확보 및 병목 해소

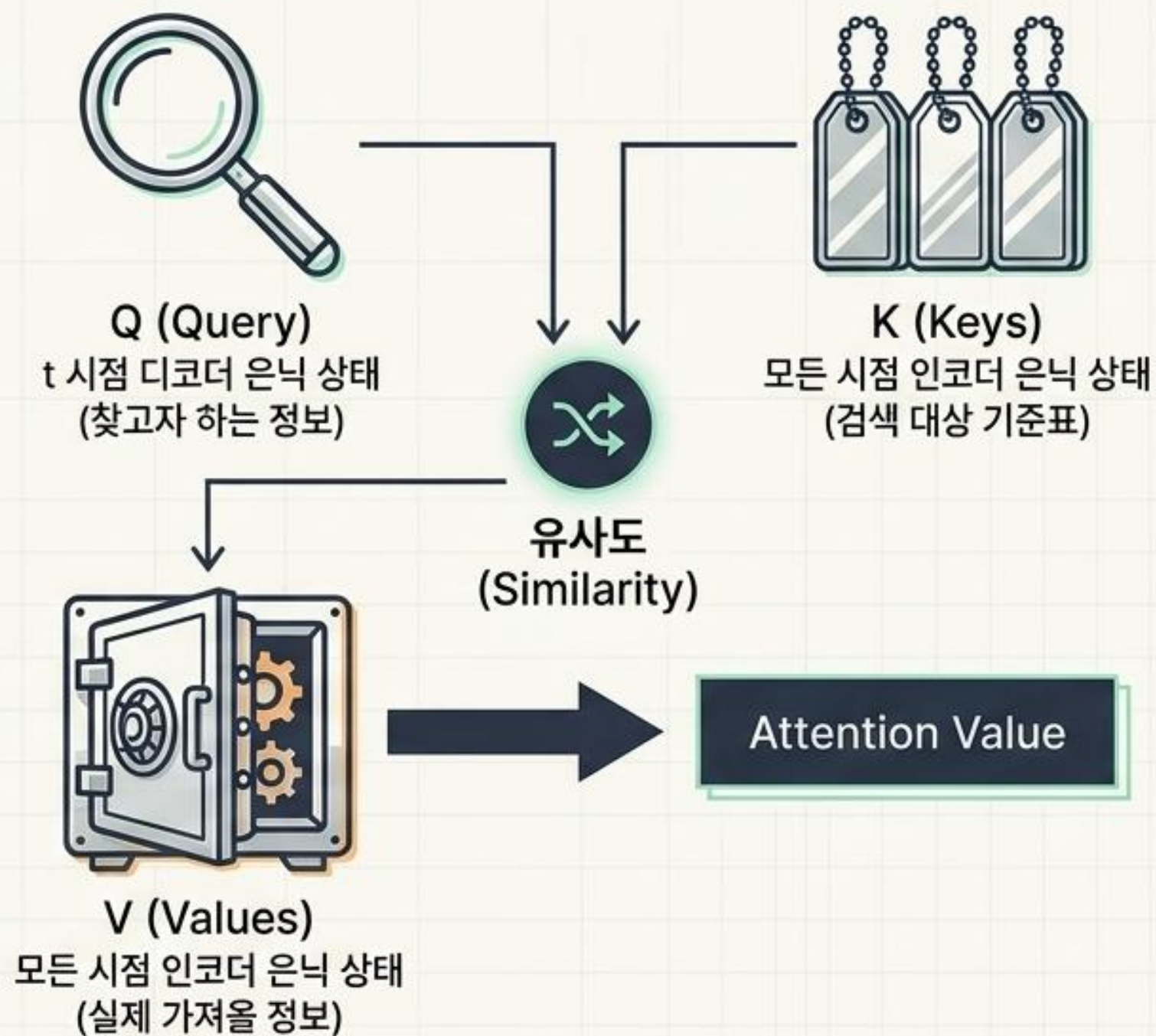


핵심 원리: Query, Key, Value의 개념과 비유

```
dict = {'2017': 'Transformer',  
        '2018': 'BERT'}  
  
print(dict['2017'])  
  
-> Transformer
```

어텐션 연산 논리:

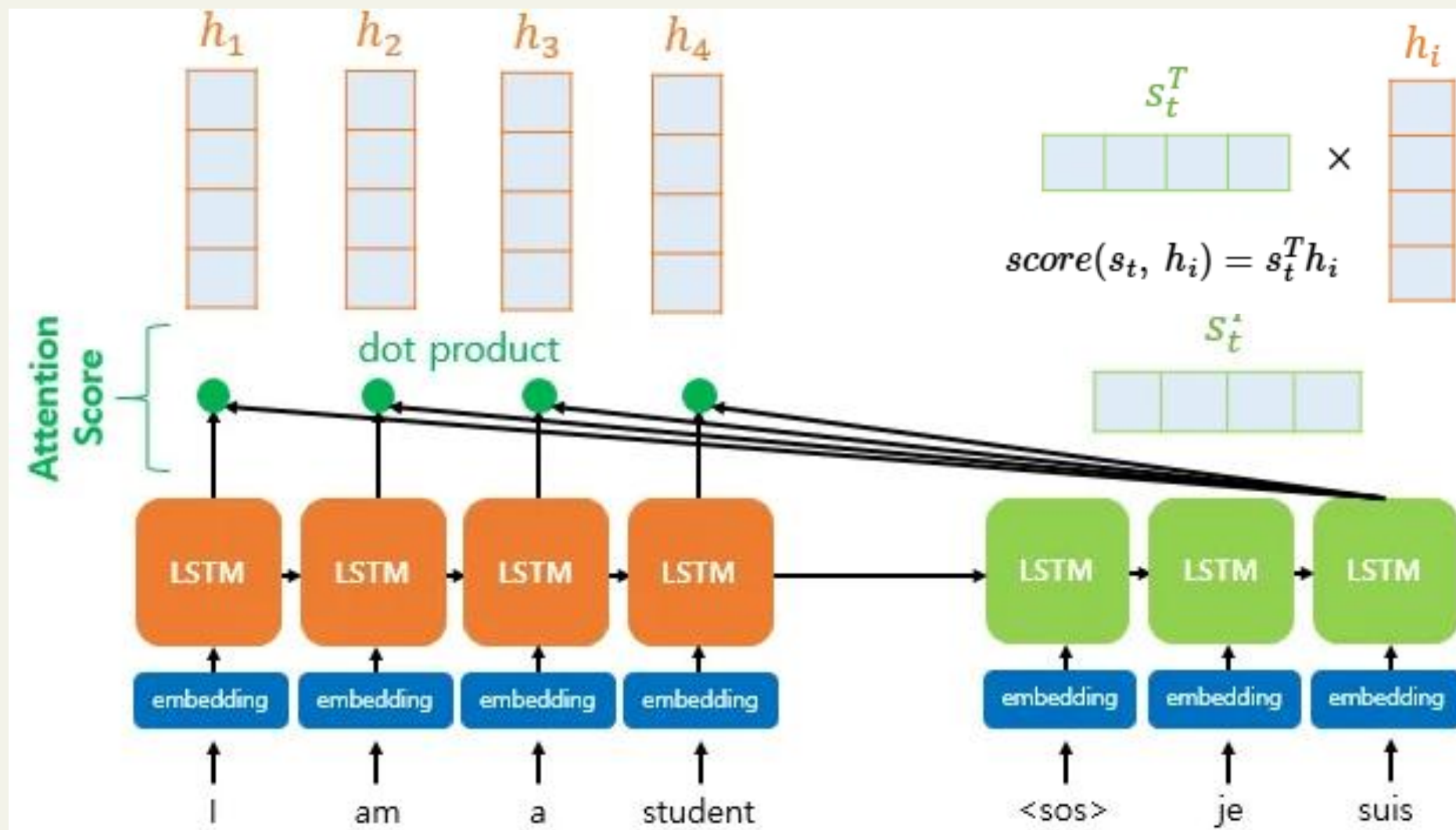
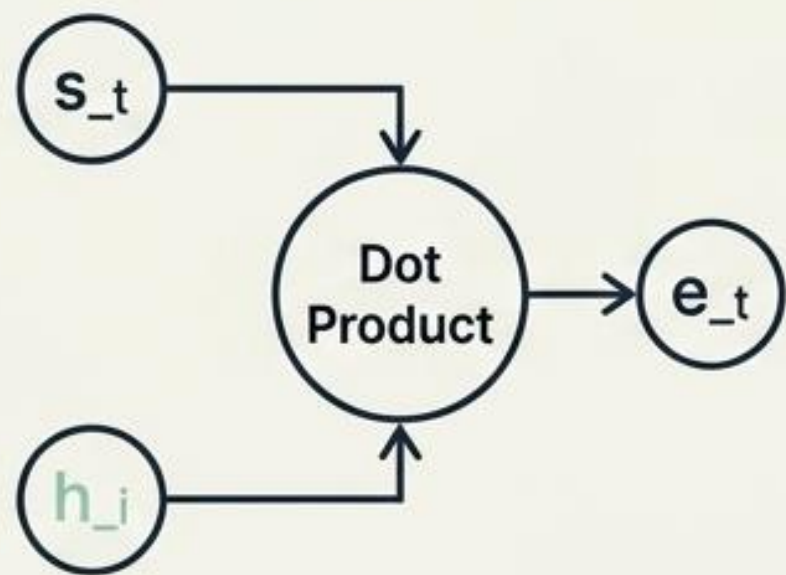
$\text{Attention}(Q, K, V) = \text{Attention Value}$



닷-프로덕트 어텐션 프로세스 해체 (Step 1~3)

Step 1: 어텐션 스코어 산출

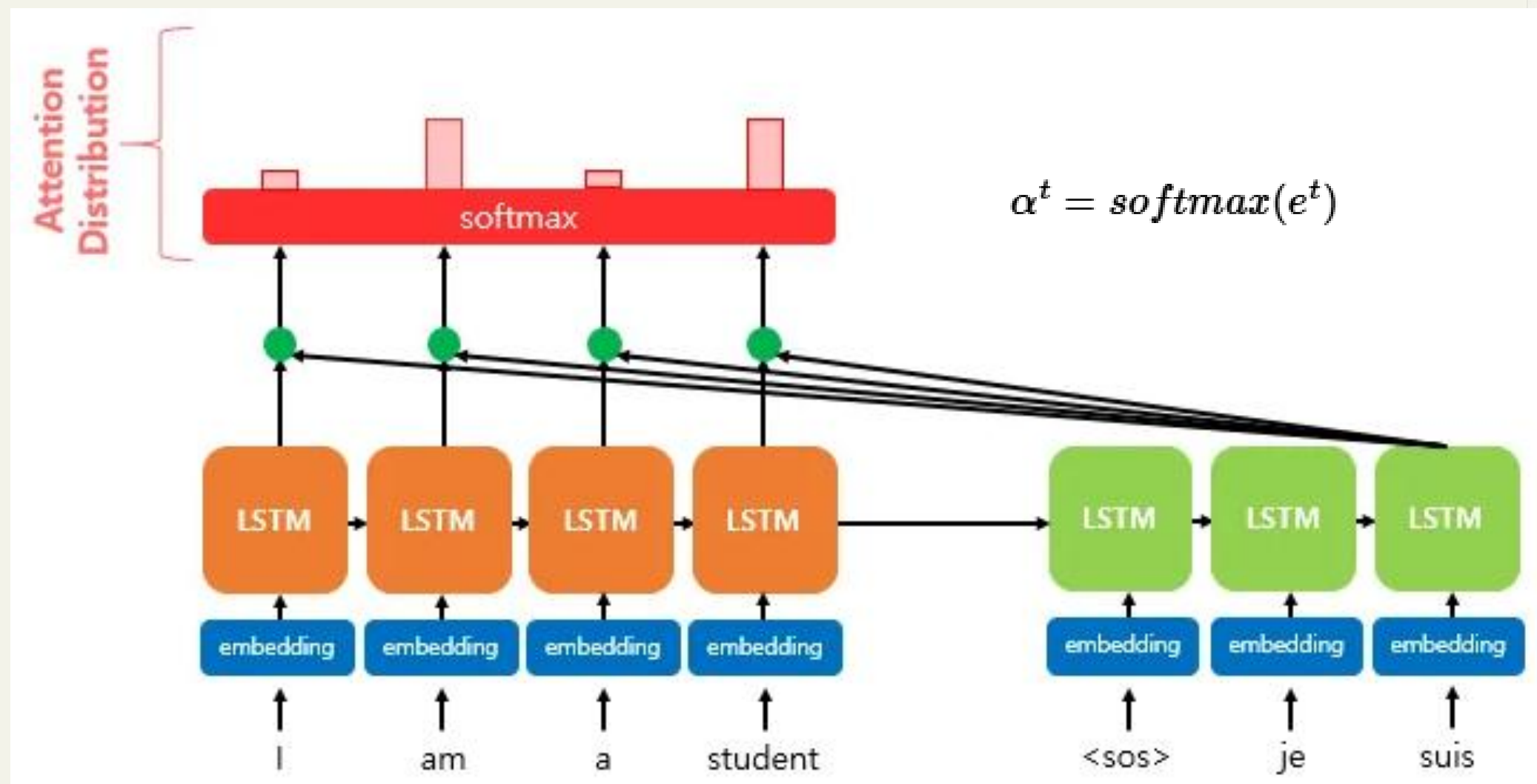
- 디코더 상태(s_t)와 인코더 상태(h_i)의 내적(Dot Product)
- 스코어 모음값 e_t (스칼라 유사도) 산출



닷-프로덕트 어텐션 프로세스 해체 (Step 1~3)

Step 2: 소프트맥스 어텐션 분포

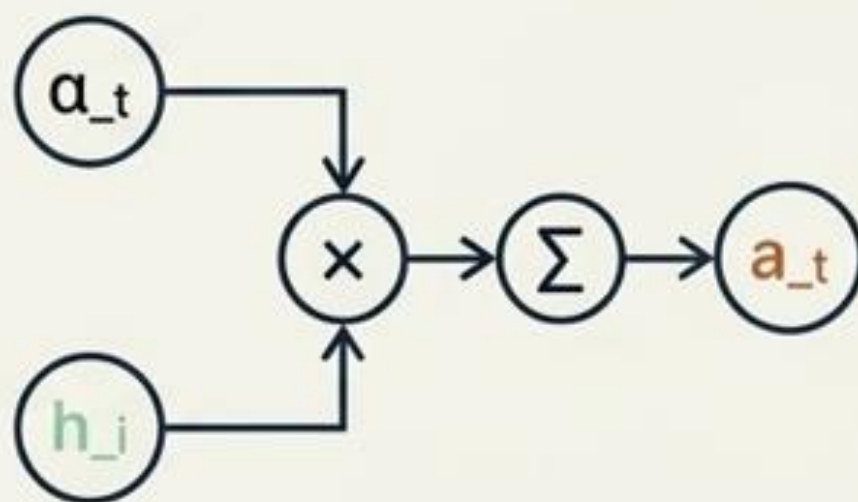
- e_t 에 Softmax 적용하여 합이 1인 확률 분포 도출
- 어텐션 가중치(α_t) 획득



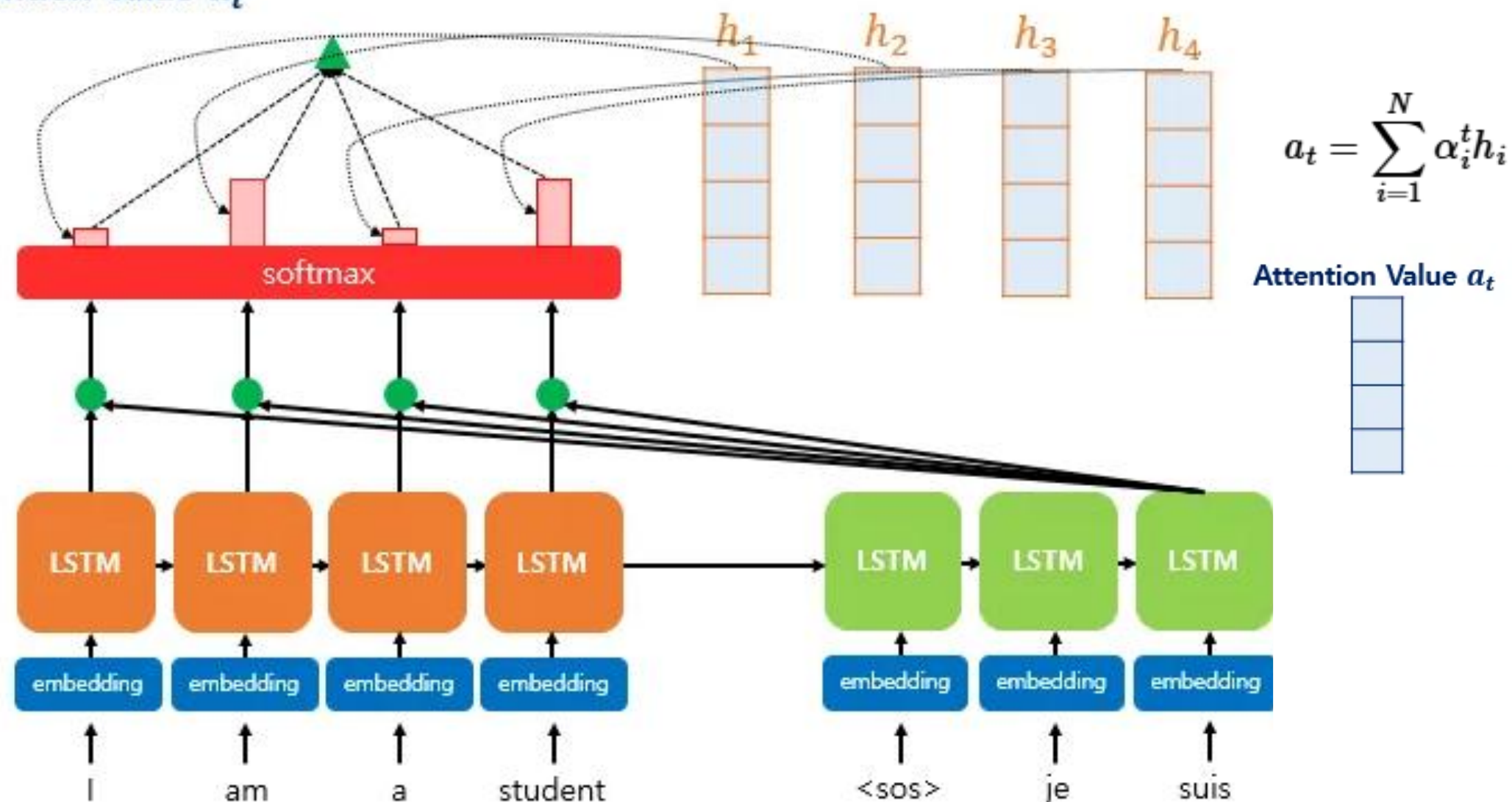
닷-프로덕트 어텐션 프로세스 해체 (Step 1~3)

Step 3: 가중합 (컨텍스트 벡터)

- h_i 와 a_t 를 곱하여 모두 더함 (Weighted Sum)
- 압축된 문맥 정보인 **Attention Value** (a_t) 도출



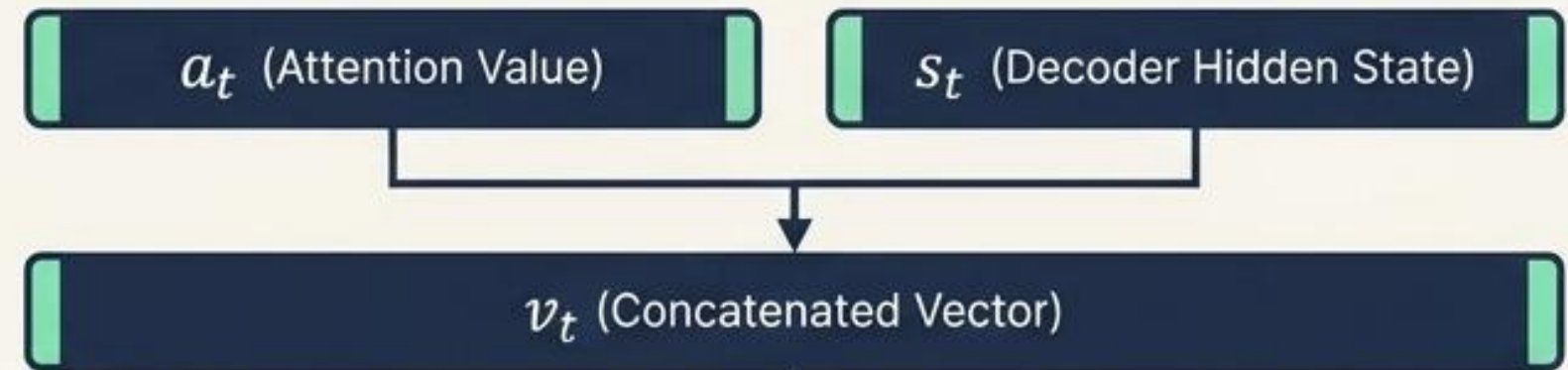
Attention Value a_t



닷-프로덕트 어텐션 프로세스 해체 (Step 4~6)

Step 4: 상태 결합 (Concatenate)

- 어텐션 값(a_t)과 현재 디코더 은닉 상태(s_t)를 결합하여 벡터(v_t) 생성



Step 5: 출력층 입력 연산

- v_t 에 가중치(W_c) 곱셈 후 tanh 활성화 함수 통과
- 새로운 벡터($\tilde{s}_t$ 킬다) 산출

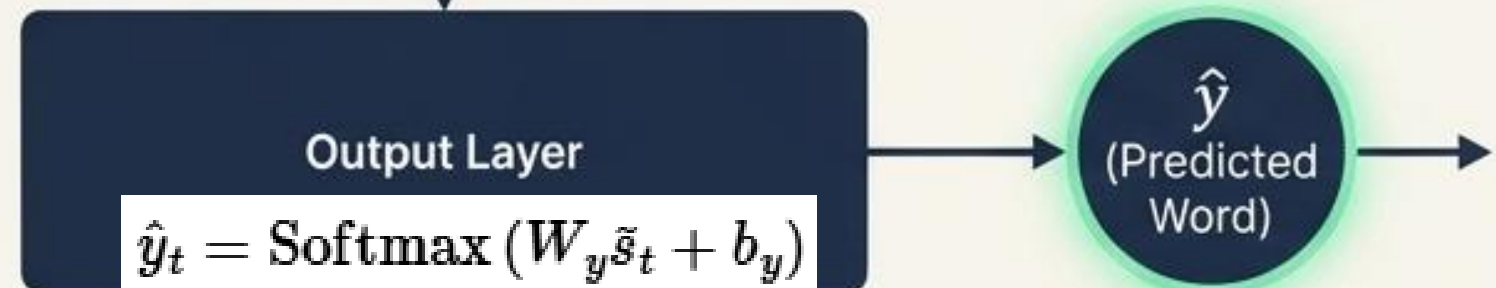


$$\tilde{s}_t = \tanh(\mathbf{W}_c[a_t; s_t] + b_c)$$

Activation

Step 6: 단어 예측 연산

- $\tilde{s}_t$ 킬다를 출력층(Output Layer)에 입력하여 최종 예측 단어($\hat{y}$ 헛) 도출

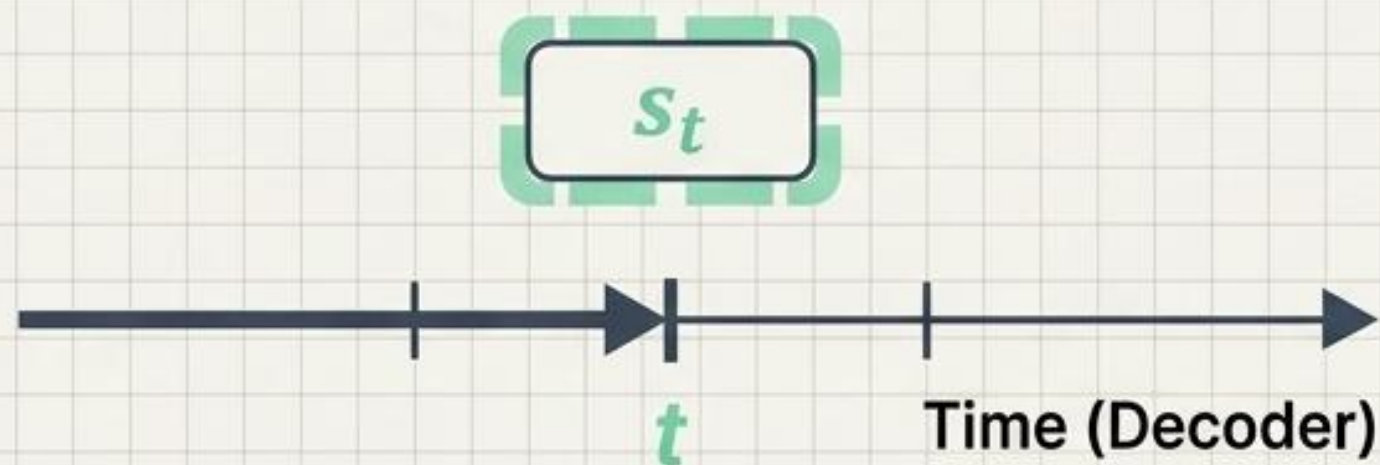


어텐션 스코어 함수의 진화 및 분류

연산 목적과 효율성에 따른 중간 수식(Score Function)의 다변화

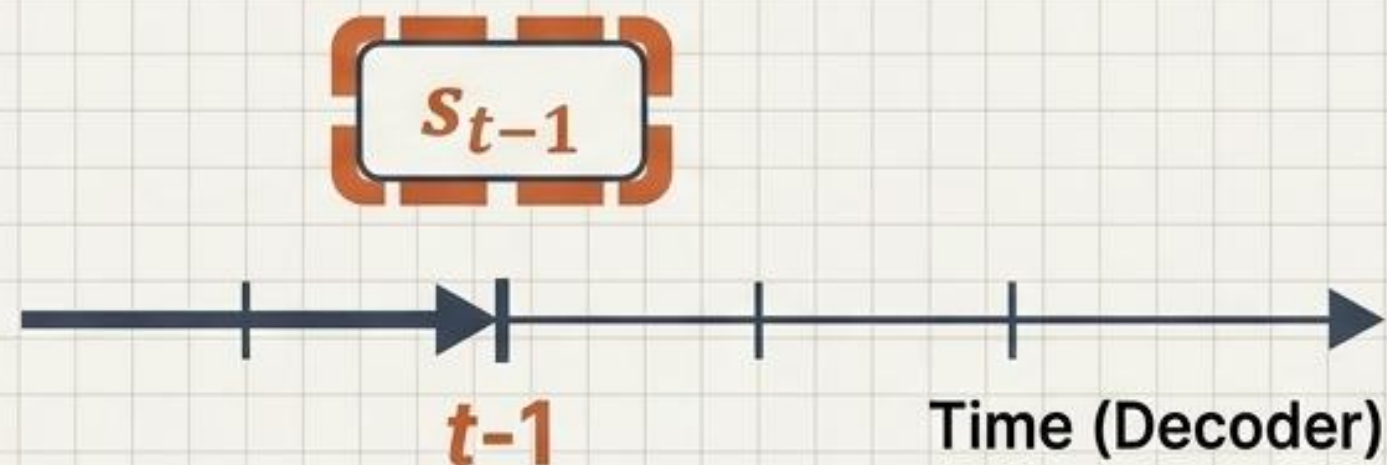
이름 (Name)	스코어 함수 (Score Function)	제안자 (Defined by)
Dot	$\text{score}(\mathbf{s}_t, \mathbf{h}_i) = \mathbf{s}_t^T \mathbf{h}_i$	Luong et al. (2015)
Scaled dot	$\text{score}(\mathbf{s}_t, \mathbf{h}_i) = \frac{\mathbf{s}_t^T \mathbf{h}_i}{\sqrt{n}}$	Vaswani et al. (2017)
General	$\text{score}(\mathbf{s}_t, \mathbf{h}_i) = \mathbf{s}_t^T \mathbf{W}_a \mathbf{h}_i$	Luong et al. (2015)
Concat	$\text{score}(\mathbf{s}_t, \mathbf{h}_i) = \mathbf{W}_a^T \tanh(\mathbf{W}_b \mathbf{s}_t + \mathbf{W}_c \mathbf{h}_i)$	Bahdanau et al. (2015)
Location-base	$\alpha_t = \text{softmax}(\mathbf{W}_a \mathbf{s}_t)$	Luong et al. (2015)

Luong (Dot-Product)



- Query 기준 시점: 디코더의 현재 시점 (t)
은닉 상태 s_t 활용
- 스코어 계산: 단순한 내적(Dot Product)
연산 위주로 빠르고 직관적 구조

Bahdanau (Concat)



- Query 기준 시점: 디코더의 이전 시점 ($t-1$)
은닉 상태 s_{t-1} 활용 (선제적 문맥 참조)
- 스코어 계산: 가중치 행렬 도입 및 tanh 함수
기반의 복잡한 신경망 연산 수행
- 병렬 처리: 은닉 상태를 행렬 H로 묶어 연산
효율성 도모

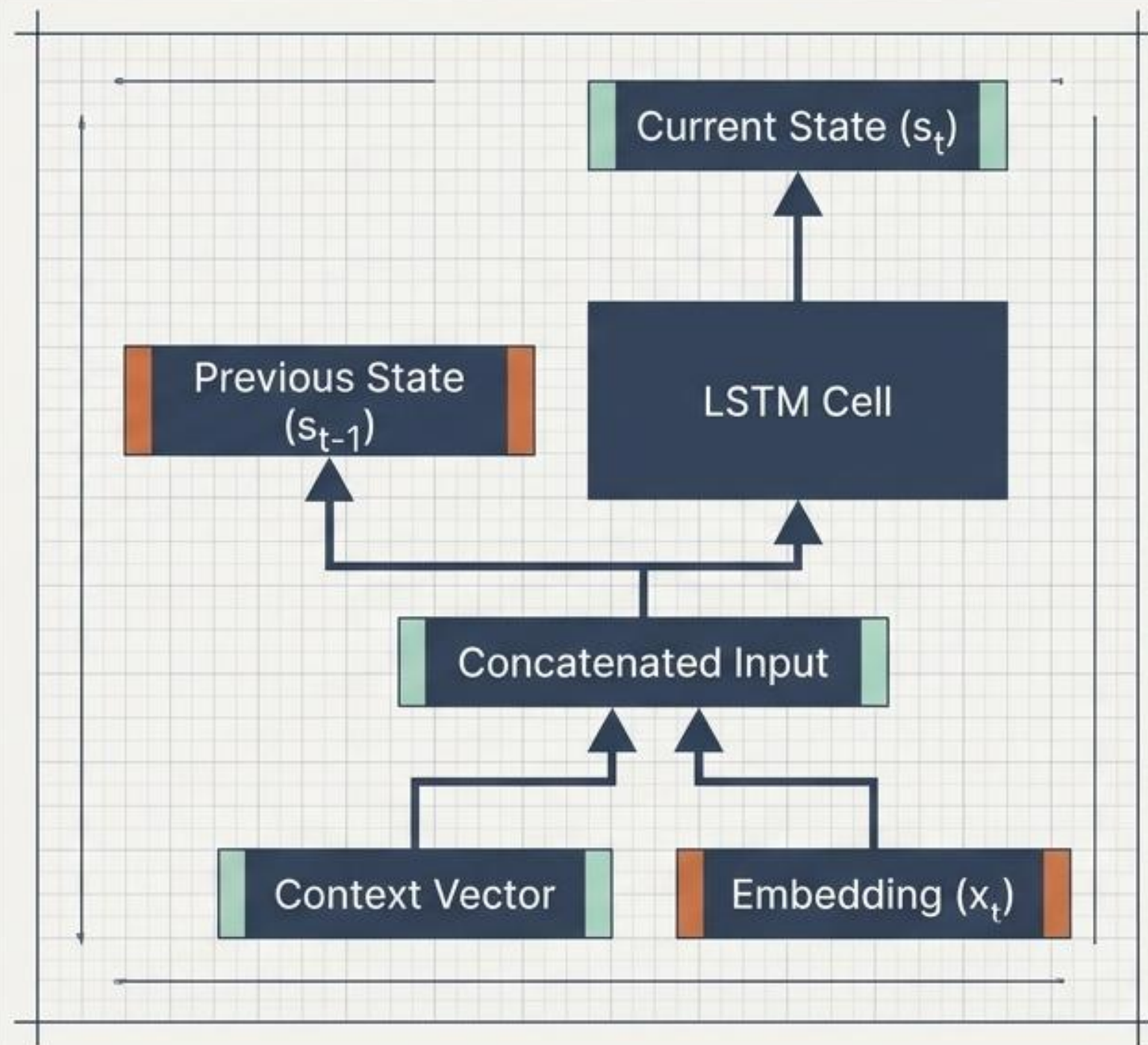
바다나우 어텐션의 데이터 파이프라인

혁신적 입력 결합 (Concatenate)

- 기존: 이전 은닉 상태(s_{t-1})와 현재 임베딩 단어(x_t)만 연산에 사용
- 바다나우: '컨텍스트 벡터'와 '임베딩 단어 벡터'를 결합하여 완전히 새로운 입력값 생성

시퀀스 예측 성능 강화

- 인코더의 전체 문맥 정보가 디코더 입력 레이어부터 직접적 영향을 미치도록 설계



파이토치(PyTorch) 실전 구현 1: Encoder 아키텍처

- 모델 초기화: 임베딩 256, 은닉 상태 256
- 순방향 전파: 어텐션을 위해 단일 벡터가 아닌 모든 시점의 은닉 상태(outputs)를 디코더로 반환
- 핵심 리턴값: outputs, hidden, cell

```
class Encoder(nn.Module):  
    def __init__(self, src_vocab_size, embedding_dim,  
hidden_units):  
        super(Encoder, self).__init__()  
        self.embedding = nn.Embedding(src_vocab_size,  
embedding_dim,  
padding_idx=0)  
        self.lstm = nn.LSTM(embedding_dim,  
hidden_units, batch_first=True)  
  
    def forward(self, x):  
        x = self.embedding(x)  
        outputs, (hidden, cell) = self.lstm(x)  
        return outputs, hidden, cell
```


파이토치(PyTorch) 실전 구현 2: Decoder 아키텍처

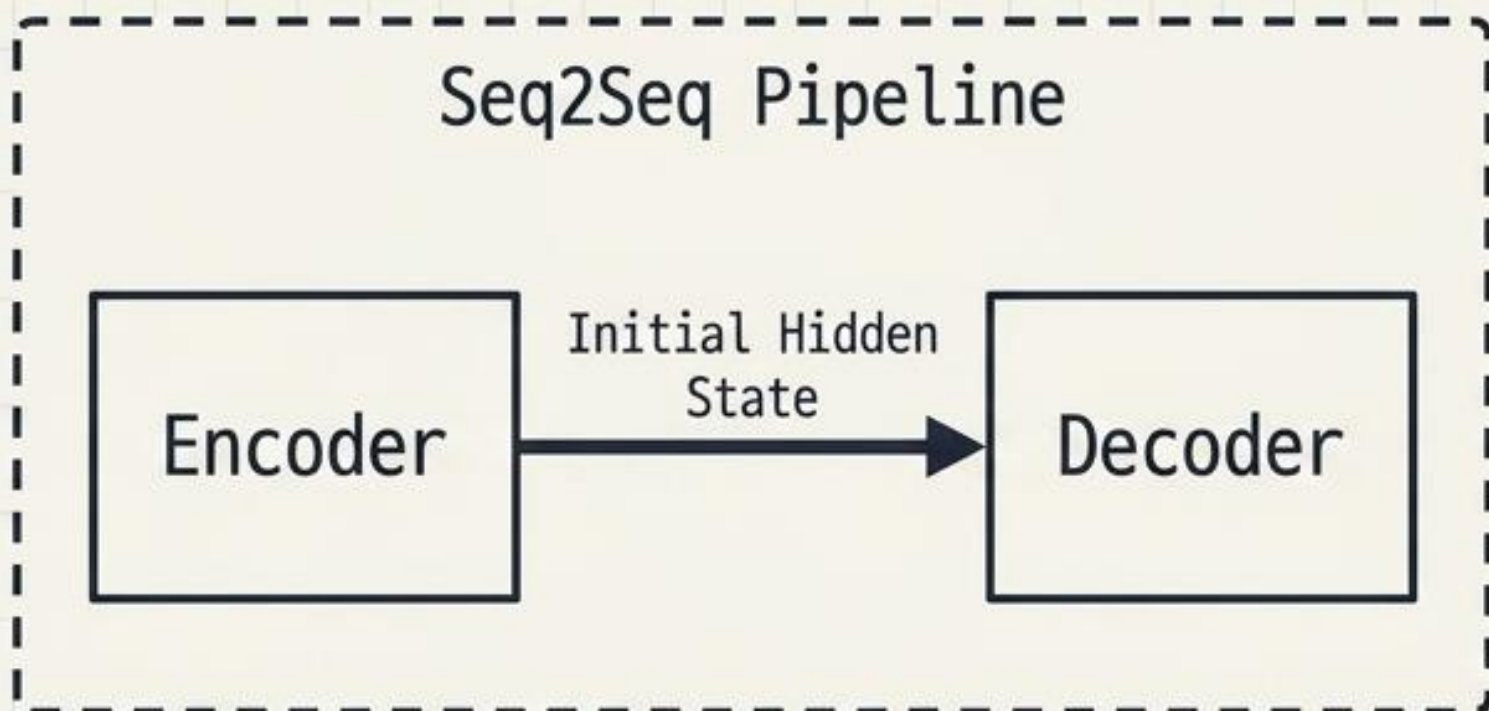
주요 개념 (Key Concepts)

- **어텐션 스코어**: torch.bmm을 사용한 배치 행렬 내적 연산 수행
- **가중치 및 벡터 생성**: Softmax를 거친 가중치를 다시 인코더 아웃풋과 곱해 컨텍스트 벡터 획득
- **병합**: torch.cat으로 컨텍스트 벡터와 임베딩 입력 병합

```
class Decoder(nn.Module):
    # ... (init omitted) ...
    def forward(self, x, encoder_outputs, hidden, cell):
        x = self.embedding(x)
        # Attention Score calculation
        attention_scores = torch.bmm(
            encoder_outputs,
            hidden.transpose(0, 1).transpose(1, 2)
        )
        attention_weights = self.softmax(attention_scores)
        context_vector = torch.bmm(
            attention_weights.transpose(1, 2),
            encoder_outputs
        )
        # Repeat and Concatenate
        seq_len = x.shape[1]
        context_vector_repeated = context_vector.repeat(1, seq_len, 1)
        x = torch.cat((x, context_vector_repeated), dim=2)
        output, (hidden, cell) = self.lstm(x, (hidden, cell))
        output = self.fc(output)
        return output, hidden, cell
```

Code adheres to PyTorch best practices for attention implementation.

파이토치(PyTorch) 실전 구현 3: Seq2Seq 통합



- 손실 함수: 패딩 제외
`nn.CrossEntropyLoss(ignore_index=0)`
다중 클래스 분류
- 최적화: 안정적 수렴을 위한 `optim.Adam` 사용

```
class Seq2Seq(nn.Module):
    def __init__(self, encoder, decoder):
        super(Seq2Seq, self).__init__()
        self.encoder = encoder
        self.decoder = decoder

    def forward(self, src, trg):
        encoder_outputs, hidden, cell =
self.encoder(src)
        output, _, _ = self.decoder(trg,
encoder_outputs, hidden, cell)
        return output

encoder = Encoder(src_vocab, 256, 256)
decoder = Decoder(trg_vocab, 256, 256)
model = Seq2Seq(encoder, decoder)
loss_function = nn.CrossEntropyLoss(ignore_index=0)
optimizer = optim.Adam(model.parameters())
```

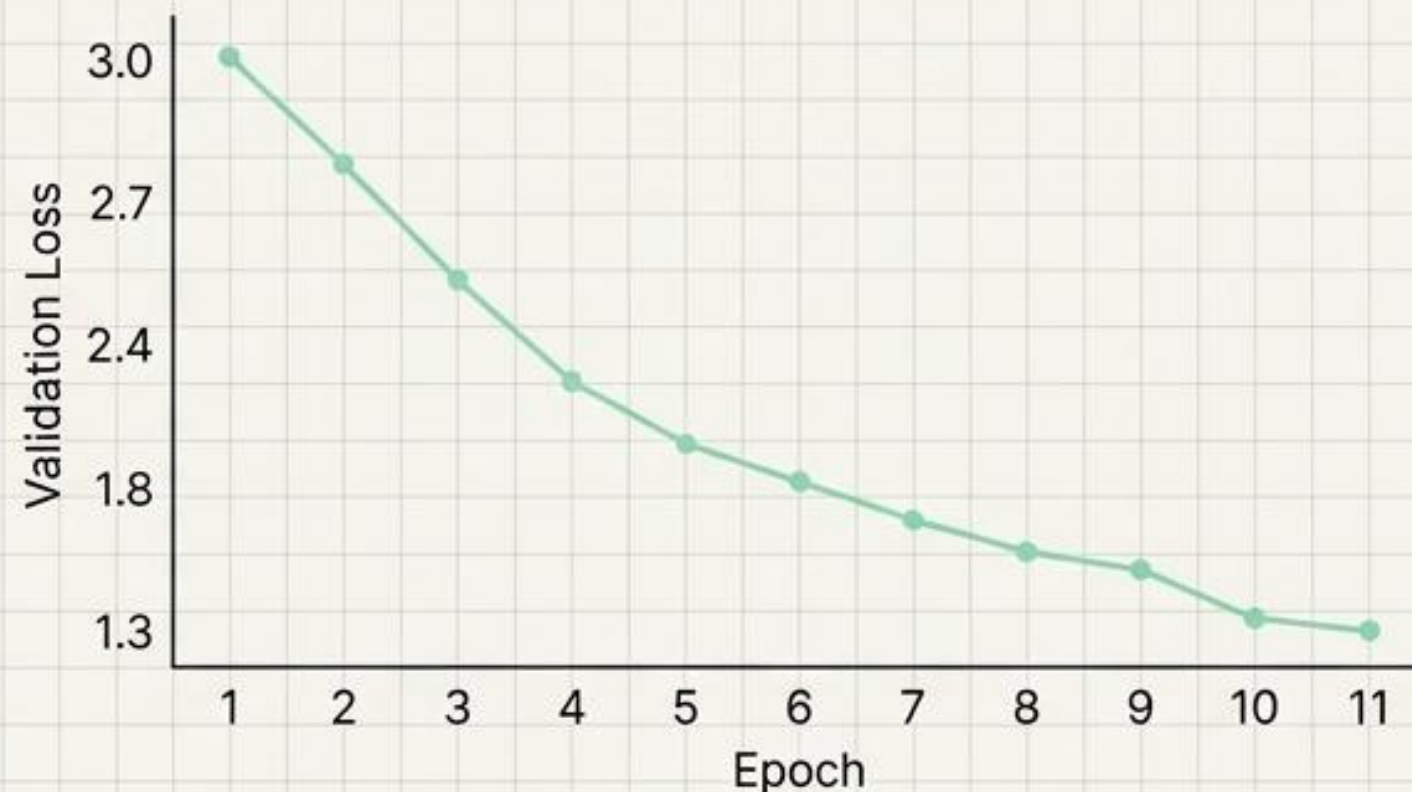

검증 및 최적화: 학습 과정 분석 및 지표

> Training initialized... (30 Epochs)

Epoch 1: Train Loss 2.9015 | Valid Loss 3.0118
Validation loss improved. Checkpoint saved.

...

Epoch 10: Train Loss 0.3790 | Valid Loss 1.3990
Epoch 11: Train Loss 0.3790 | Valid Loss 1.3990
Epoch 11: Train Loss 0.3285 | Valid Loss 1.3973
Validation loss improved to 1.3973. Checkpoint saved.



최종 도출 성능 (Best Model Checkpoint):

- Validation Loss: 1.3973
- Validation Accuracy: 0.7334 (73.3% 단어 예측 정확도)

결과 검증 1: 추론 메커니즘과 훈련 데이터 번역

- 교사 강요(Teacher Forcing) 배제: t 시점 예측 단어가 $t+1$ 의 입력으로 자체 순환되는 진정한 추론 사이클 구현

원문:	let s keep it .
정답:	gardons cela comme ca
예측:	gardons le .

원문:	how was the trip ?
정답:	comment fut votre voyage ?
예측:	comment etait le voyage ?

원문:	is it dangerous ?
정답:	est ce dangereux ?
예측:	est ce dangereux ?

결과 검증 2: 미지의 데이터 일반화 성능 및 결론

원문: that s no good .

번역: ce n est pas bon .

원문: nobody was home .

번역: personne n etait chez nous .

원문: look out !

번역: **attention !**

결론: 정보 손실과 기울기 소실을 극복한 어텐션 메커니즘은 단순한 보조 장치를 넘어, '트랜스포머(Transformer)' 시대를 여는 핵심 패러다임으로 진화하였습니다.